Trabajo Práctico 8: Transcriptómica y GSEA

Introducción y Objetivos

Este trabajo práctico tiene por objetivo familiarizar al alumno con algunos de los análisis más estándar que se hacen con datos transcriptómicos una vez obtenidos los genes diferencialmente expresados (DEGs, "Differentially Expressed Genes"). Ya se trate de microarrays, bulk RNAseq o single-cell RNAseq, la etapa de control de calidad de datos y obtención de DEGs a partir de los datos crudos se realiza típicamente en R mediante libraries como limma, DESeq2 o Seurat y al igual que en le procesamiento de datos de NGS se encuentra estandarizado de acuerdo al origen de los datos y método experimental elegido.

En este trabajo práctico se realizarán algunos de los gráficos más convencionales a partir de los DEGs (PCA, heatmap y volcano plot) y luego se realizarán dos análisis funcionales típicos: Pathway Enrichment Analysis y Gene-Set Enrichment Analysis (GSEA). Es importante notar que es a partir de estos analisis como punto de partida donde suele aparecer el ingenio y la creatividad del investigador y la necesidad de utilizar MAS bioinformática como aprendida en la materia. En el caso de que esten interesados se los dejamos para el TP especial.

Objetivo 1. Preparación y gráficos iniciales

Descarga y exploración de los datos

En primer lugar tendrá que descargar dos tablas: "DE_table.csv" y "full_matrix.csv". Estas corresponden a un experimento de microarray de Affymmetrix en el que se comparan tumores tiroideos papilares (PTC, "Papillary Thyroid Carcinoma") con tejido sano de tiroides (WT). La tabla "DE_table.csv" contiene el resultado de la expresión diferencial para todos los genes. Mirelo y explore qué columnas tiene. La primera no tiene nombre y corresponde a los "row names", son concretamente los ID de las sondas del array.

- **logFC**: (*log fold change*) es el logaritmo en base 2 del cociente entre la media de la expresión de cada condición para esa sonda. su signo indica en cuál condición está sobre/sub - expresado.
- **AveExpr**: es la media de la expresión de todas las muestras para esa sonda
- **t**: el valor de t-student que se usó para la prueba estadística de la sonda en cuestión
- **P.Value**: es el p-valor no ajustado de la prueba estadística para esa sonda
- **adj.P.Val**: p-valor ajustado para contrastes múltiples mediante el método de FDR (False Discovery Rate) de la prueba estadística
- **Symbol, Refseq, EntrezID, GeneName, Ensembl**: identificadores para el gen correspondiente a la sonda del array

La segunda tabla, "full_matrix.csv" contiene los valores de expresión normalizados de cada sonda para cada muestra y será de utilidad a la hora de hacer el PCA y el heatmap.

En este trabajo, además de usar las libraries típicas como *pandas*, *numpy* y

matplotlib, se necesita también "*gseapy*", "*seaborn*" y "*scikit-learn*". Para ello tiene que instalarlas con "`pip3 install gseapy`", "`pip3 install seaborn`" y "`pip3 install scikit-learn`".

PCA, Heatmap y Volcanoplot

Antes que nada hace falta hacer un filtrado de la tabla DE_table usando algún criterio para quedarse **"solo"** con aquellos genes que se puedan considerar significativamente diferencialmente expresados. Habitualmente se suele establecer cortes usando el p-valor ajustado, el |logFC| o una combinación de ambos. Para el primer caso lo más común es quedarse con aquellos que tengan p-valor < 0.05. Para el segundo caso existen varios cortes comunes: |logFC| > X, donde X suele ser 0.58 (1.5 veces más expresado en una condición que la otra), 1 (2 veces más) o 1.5 (3 veces más). La elección de cuál de los dos filtros (o ambos) usar, así como el valor de corte, es una decisión condicionada por el experimento y los objetivos del investigador. Es fácil darse una idea de con cuantos DEGs uno se queda con uno o ambos cortes cargando los datos en un dataframe y haciendo un poco de magia pytonesca.

Ejercicio 1- Genere un nuevo dataframe a partir de "DE_table" que contenga solo aquellas filas que satisfagan algún criterio de corte a su elección. ¿Cuántos DEGs resultaron de su filtrado?

Una manera muy útil de ver de qué modo influyen estos cortes es viendo gráficamente la dispersión de los genes en función del logFC y el p-value. Para esto es de suma utilidad el gráfico de volcano plot. Este representa cada gen con un punto (*scatter plot*), el eje Y es el log10(p-value) el eje X es el logFC. De este modo se puede saber cuántos genes se incluyen o dejan afuera según los cortes elegidos y qué cortes pueden influir más.

2- Implemente un volcanoplot para representar todos los puntos de la tabla original "DE_table". Puede añadir una línea horizontal al gráfico si eligió un corte por p-value o dos líneas verticales para el corte de logFC positivo y negativo si eligió logFC para filtrar. Incluso si lo desea puede colorear los puntos de color distinto según el cuadrante en que caigan. ¿Cómo es la dispersión de los puntos? ¿Están más esparcidos por algunos de los dos ejes? ¿Le parece que hizo un filtrado laxo o astringente?

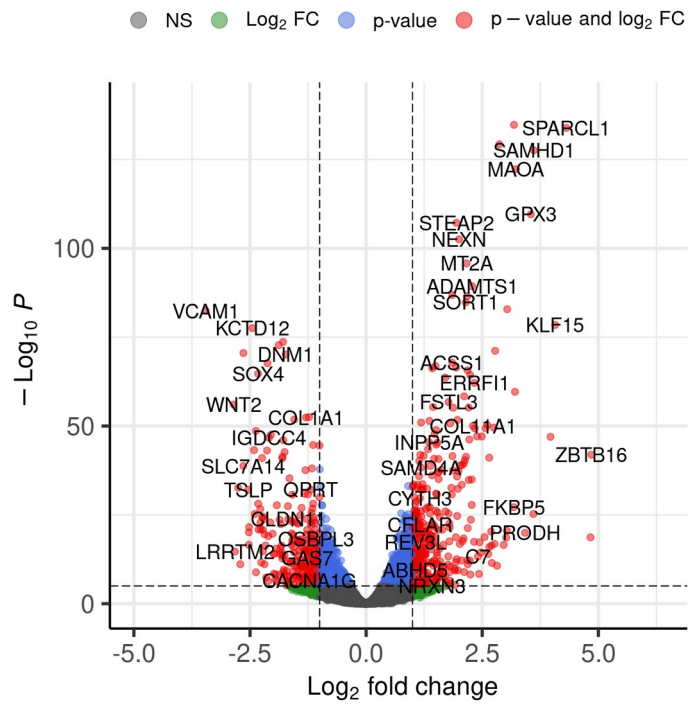


Fig 1. Ejemplo de volcanoplot.

Otro gráfico bastante habitual y que sirve para ver la manera en que se relacionan las muestras del experimento y/o la influencia de los tratamientos/condiciones experimentales es el PCA (“Principal Components Analysis”), que es un método de reducción de la dimensionalidad. En resumidas cuentas, utiliza álgebra linear para condensar la información datos multidimensionales en menos dimensiones (los componentes principales), de manera que puedan ser visualizados en un gráfico bidimensional (usualmente se grafican los dos primeros componentes principales (ya que es dificil hacer/interpretar gráficos en más de 2 dimensiones), que son los que explican la mayor parte de la variabilidad de los datos).

3- Utilice los DEGs que filtró mediante el p-value o el logFC para generar un gráfico de PCA. Para ello necesita el paquete `scikit-learn`. Además, tendrá que subsetear la matriz “full_matrix.csv” utilizando los DEGs con los que se quedó tras filtrar por logFC/p-value. De modo que el PCA se genere solo con los datos de expresión normalizados de los DEGs.

Le dejamos un poco de codigo para ello:

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
scaled_matrix = sc.fit_transform(matriz_DEGs) #centra y escala la
matriz
pca = PCA(n_components=2)
pca_res = pca.fit(scaled_matrix) #“fitea” PCA a la matriz
```

Una vez genere el PCA tiene que graficarlo con `plt.scatter()`. ¿Qué tal es la separación de las muestras?

```
colors = ["red", "blue"]*9 #Esto es para colorear los puntos por condicion
plt.figure(figsize=(8, 6))
plt.scatter(pca_res.components_[0, :], pca_res.components_[1,:], color=colors)
```

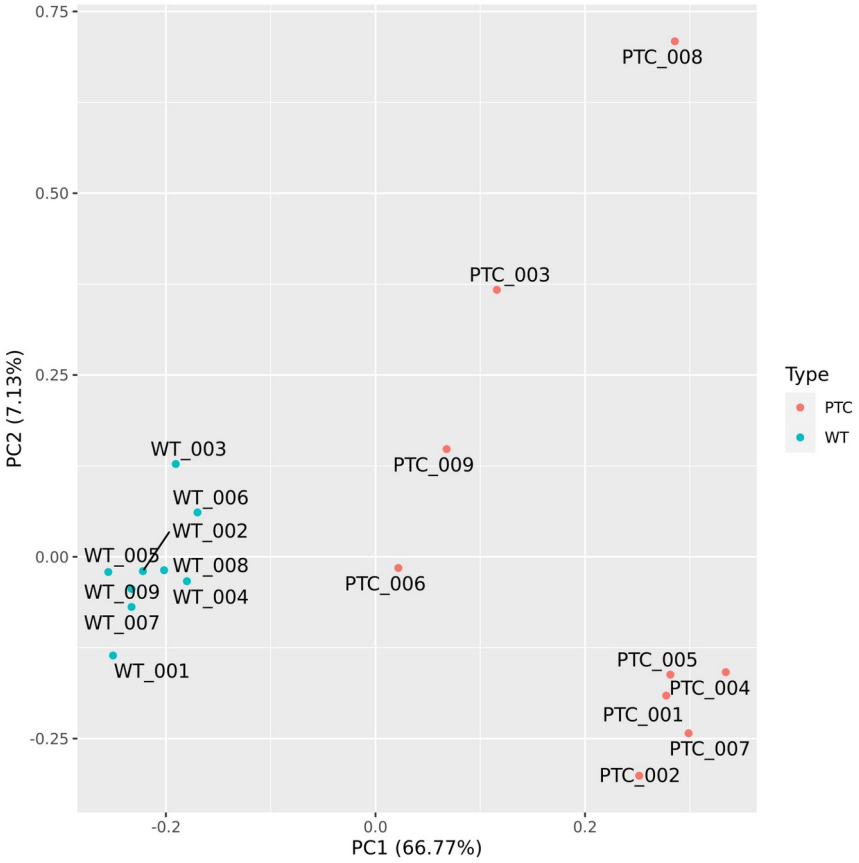


Fig 2. Ejemplo de PCA plot.

El atributo "`components_`" del objeto PCA le permite acceder a las coordenadas de cada punto en cada PC. También puede conocer el porcentaje de varianza explicada por cada PC con el atributo "`explained_variance_ratio_`". ¿Cuánta varianza está explicada solo en el PC1?

4- Por último el heatmap. Para esto va a necesitar "`seaborn`" y la matriz de datos de expresión subseteada que ya usó en el PCA.

```
import seaborn as sns
```

```
sns.clustermap(matriz_DEGs, method='complete', cmap='RdBu_r',
figsize=(8, 6), z_score=0)
```

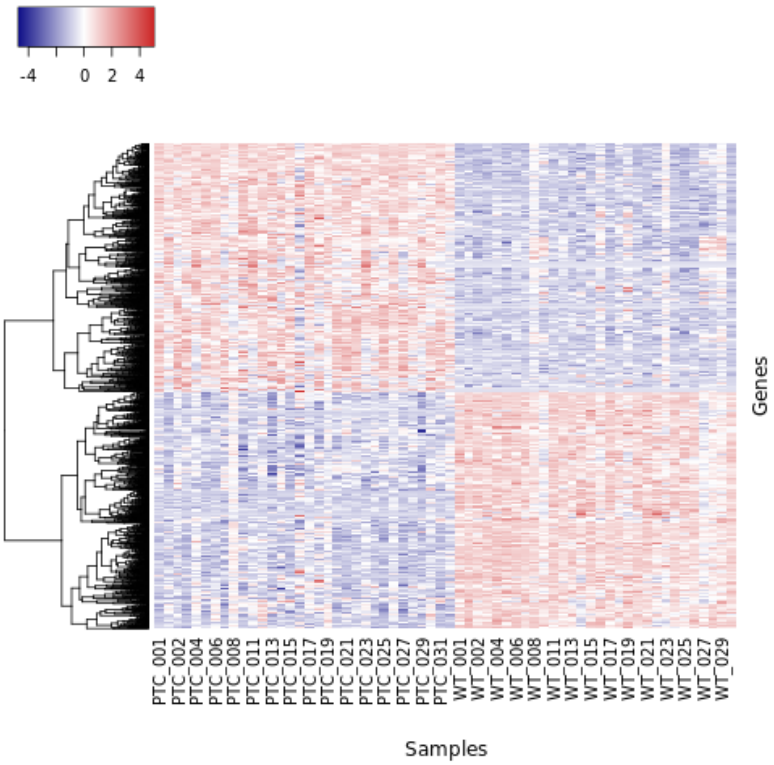


Fig 3. Ejemplo de heatmap.

Objetivo 2. Pathway Enrichment Analysis (PEA) y Gene-Set Enrichment Analysis (GSEA)

En esta sección nos adentramos en el análisis más funcional de los datos transcriptómicos. PEA y GSEA son dos métodos similares que buscan analizar si los DEGs pertenecen de manera significativa a ciertas vías metabólicas y/o de señalización y/o de funciones celualres/moleculares específicas. Para esto se utilizan bases de datos que contienen listas de genes que se describieron como asociados a alguna vía en particular, como Gene Ontology (GO), KEGG, Reactome, MSigDB, etc…

Pathway Enrichment Analysis (PEA)

1- Importe "`gseapy`" y corra la función "`get_library_name()`". Eche un vistazo a lo que devuelve. Elija algunas bases de datos para usar durante el resto del trabajo práctico. Las más usuales son las mencionadas antes: GO (*Molecular Function*, *Biological Process* y *Cellular Component*), KEGG, Reactome y MSigDB. Recuerde que estamos trabajando con datos humanos.

```
import gseapy as gp
gp.get_library_name(organism="Human")
```

2- Para correr PEA necesita los "gene symbols". Estos los puede obtener de la tabla de

expresión diferencial filtrada. Asegúrese de que todos los valores de la columna SYMBOL que extraiga estén en imprenta mayúscula y que no haya “NAs”. Además necesitará hacer una lista enumerando aquellas bases de datos que le interesa utilizar para el análisis (las que eligió en el punto anterior). Una vez tenga la lista de bases de datos y la lista de símbolos puede correr el análisis:

```
res = gp.enrichr(gene_list=lista_simbolos, gene_sets=lista_dbs,
organism="human")
```

Si corre `res.results`, esto le devuelve una tabla resumiendo el resultado del análisis. Cada fila corresponde a una vía metabólica o de señalización listada en alguna de las bases de datos que se indicaron. La columna “**Gene_set**” indica la base de datos de donde proviene la lista de genes correspondiente a esa vía, “**Term**” es el nombre de la lista de genes, “**Overlap**” es la cantidad de genes de su lista que coincidieron con la lista de la vía. Luego viene una serie de columnas con valores estadísticos, de las cuales la única particularmente relevante es “**Adjusted P-value**” (esta indica si las coincidencias de genes en la vía son atribuibles al azar o si hay un enriquecimiento verdadero en genes de esa vía) y por último “**Genes**” que es la lista de genes nuestros que coincidieron con los de la vía.

3- Mucho más práctico es visualizar el resultado del análisis gráficamente. Para esto, `gseapy` nos provee un par de funciones de graficado. ¿Nota alguna tendencia en los tipos de vías que resultaron significativas?

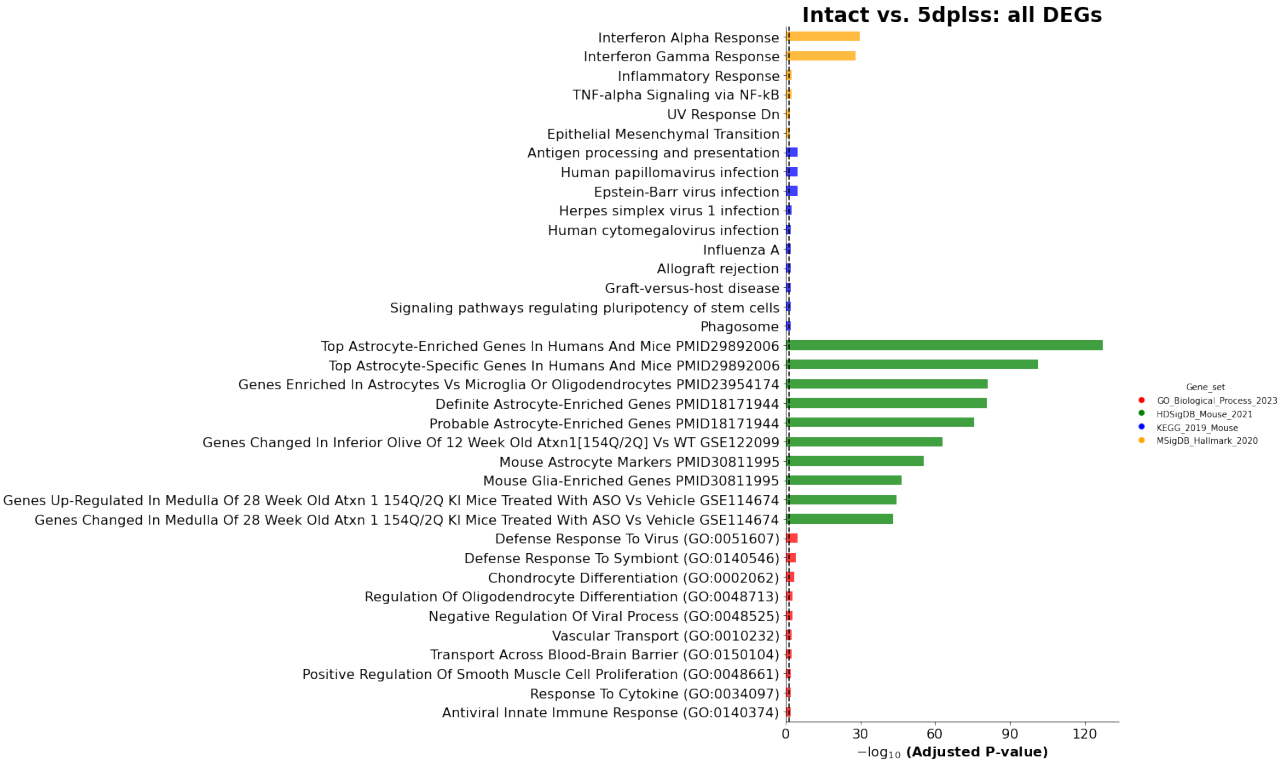


Fig 4. Ejemplo de barplot para resultados de PEA.

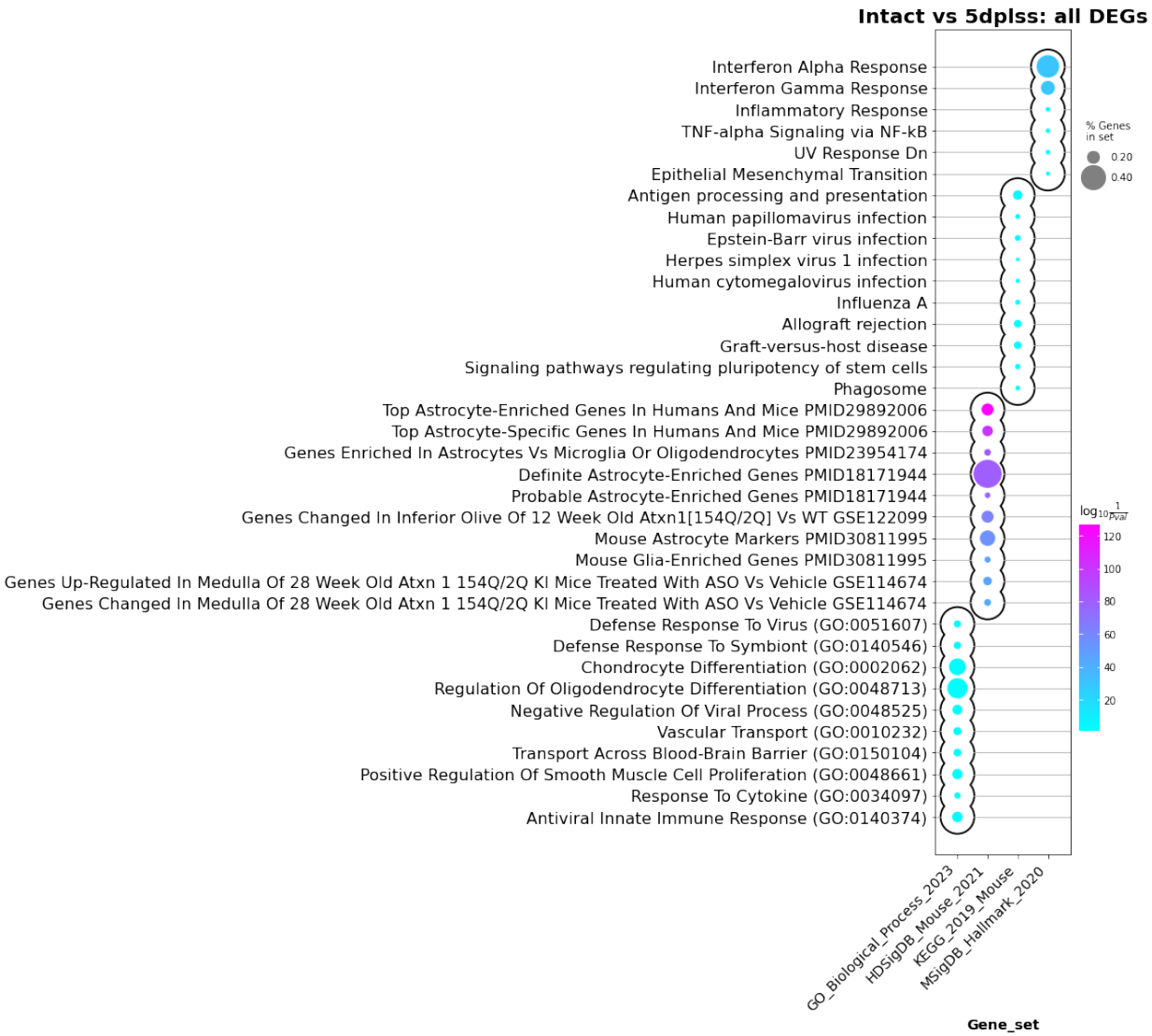


Fig 5. Ejemplo de dotplot para resultados de PEA.

```
gp.dotplot(res.results, column="Adjusted P-value", top_term = 7,
size = 5, figsize = (5, 15), x = "Gene_set", xticklabels_rot=45,
show_ring=True)

gp.barplot(enrich_res.results, column="Adjusted P-value",
group='Gene_set', size=10, top_term=7, figsize=(3,13))
```

Gene-Set Enrichment analysis (GSEA)

A pesar de la utilidad de PEA, típicamente sufre de dos problemas: no es capaz de distinguir qué genes están sobre o sub expresados en cada condición (a menos que uno lo arregle manualmente) y por ende no se puede saber qué vías están prendidas o apagadas en cada caso; y además está condicionado por los criterios de corte elegidos para los DEGs, es decir, que los resultados de PEA van a depender de cuántos y cómo hayamos seleccionado los DEGs. GSEA viene a solucionar ambos problemas. En lugar de usar los DEGs usa todos los genes y los rankea con un sistema de puntaje que combina el logFC y

el p-value. Este rankeo utiliza el signo del logFC, con lo que el sentido de la expresión (sub o sobre) está considerado. En resumen, este rankeo nos independiza del problema del filtrado de DEGs porque tiene en cuenta la magnitud del logFC y del p-value para todos los genes, y además el signo del logFC le da direccionalidad.

1- Anexe a la DE_table original una columna con el puntaje de ranking correspondiente. Este se calcula como -log10(pVal)*logFC. Luego reordene la tabla en el sentido descendiente de la columna de ranking. Genere un nuevo dataframe que contenga solo las columnas “SYMBOL” y “rank”. Asegúrese de que todos los símbolos estén en imprenta mayúscula y elimine cualquier fila que contenga NA. Este dataframe nuevo es lo que daremos de comer a la función que realiza el GSEA junto con una lista de bases de datos que queremos que utilice. Tenga en cuenta que este análisis puede tomar varios minutos, y tarda más cuantas más bases de datos elija.

```
prerank_result = gp.prerank(rnk = ranking_df, gene_sets = lista_dbs,
permutation_num = 1000, organism = "human")

prerank_result.res2d
```

Mediante el atributo “res2d” del resultado de GSEA se puede visualizar una tabla que resume los resultados. De las columnas que presenta cabe destacar “Term” que corresponde a la vía o lista de genes proveniente de base de datos, “ES” y “NES” que son “Enrichment Score” y “Normalized Enrichment Score”, que son puntajes que miden el enriquecimiento de ese término en genes sobreexpresión o subexpresados según el caso. “FDR q-val” es el p-value ajustado para esa fila y “Lead_genes” son los genes que están determinando que ese término está sobreexpresado o subexpresado.

2- Haga un gseaplot. Es un gráfico que permite visualizar para no o varios términos la distribución de su enriquecimiento en la lista rankeada de genes.

```
prerank_result.plot(terms="KEGG_2021_Human__Thyroid hormone
synthesis", show_ranking=True)
```

3- Filtre la tabla de resultados de GSEA por FDR q-val <0.05 y elija 2 términos con NES positivo y 2 con NES negativo. Guarde los nombres de los 4 términos en una lista “mis_terms”.

```
prerank_result.plot(terms=mis_terms, show_ranking=True)
```

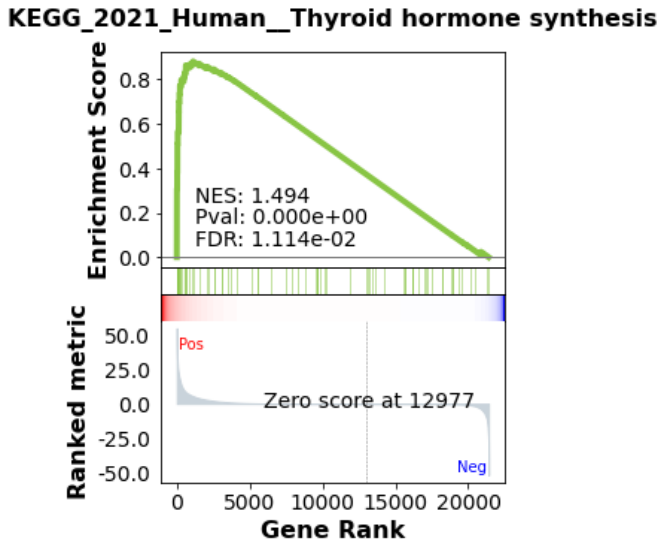


Fig 6. GSEA plot para un solo término.

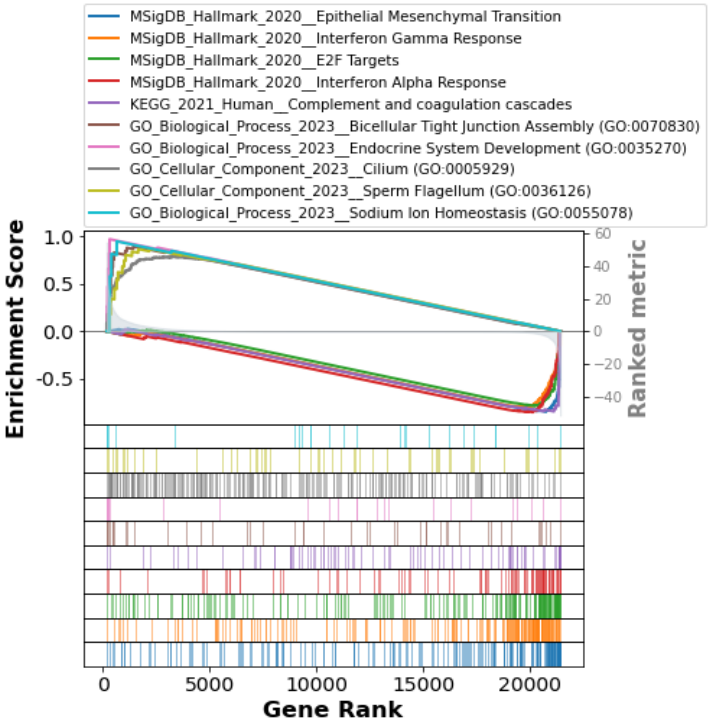


Fig 7. GSEA plot para varios términos.

4- Haga un dotplot y un barplot con los términos estadísticamente significativos. ¿Nota algún patrón en ellos?

```
dotplot(prerank_result.res2d, column="FDR q-val", size=10,
figsize=(7,14), cutoff=0.05, show_ring=True,top_term=30)

significant_terms = prerank_result.res2d[prerank_result.res2d["FDR
q-val"] < 0.05]
plt.barh(significant_terms["Term"], significant_terms["NES"])
```